```
numberSold = 0

calculateNumberSold = (salePrice) ->
  numberSold = 50 + 20/10 * (200 - salePrice)

calculateRevenue = (salePrice, callback) ->
  callback numberSold * salePrice

log = (message) ->
  console.log message
  numberSold = 'uh oh'

revenueBetween = (start, finish, callback) ->
  totals = []
  receivedResponses = 0
  expectedResponses = 0
  for price in [start..finish]
    calculateNumberSold price
    expectedResponses++
    addToTotals = (result) ->
      totals.push result
      receivedResponses++
      if receivedResponses == expectedResponses
        callback totals
      else
        log 'waiting'
    calculateRevenue price, addToTotals
```

For some unknown reason, Rob Tuse has broken your program state.

Now the program produces an array containing NaNs:

```
revenueBetween 140, 145
# [ 22400, NaN, NaN, NaN, NaN, NaN ]
```

Should you add more state to fix the problem? No. Please don't. Instead, compose the asynchronous functions, as you'll learn to do later in this chapter. Think about a program with hundreds of functions sharing hundreds of variables. How confident can you be that everything is done in exactly the correct order? How hard is it to test that they are? Avoid having variables containing state wherever possible.

How about object-oriented programming, then? After all, CoffeeScript has objects too. Object-oriented programming encapsulates state in objects. That's a different approach to functional programming. What works in